2747. Count Zero Request Servers

Daily Question - November 6, 2026

Problem Statement

You are given an integer n denoting the total number of servers and a 2D 0-indexed integer array logs, where logs[i] = [server_id, time] denotes that the server with id server_id received a request at time time.

You are also given an integer x and a 0-indexed integer array queries.

Return a 0-indexed integer array arr of length queries.length where arr[i] represents the number of servers that did not receive any requests during the time interval [queries[i] - x, queries[i]]. The intervals are inclusive.

Examples

Example 1: n = 3, logs = [[1,3],[2,6],[1,5]], x = 5, queries = [10,11]. Output: [1,2]. For [5,10], servers 1 and 2 received requests, so only server 3 is inactive. For [6,11], only server 2 received a request, so servers 1 and 3 are inactive.

Example 2: n = 3, logs = [[2,4],[2,1],[1,2],[3,1]], x = 2, queries = [3,4]. Output: [0,1]. For [1,3], all servers received at least one request. For [2,4], only server 3 received no request.

Constraints

1 <= n <= 100000; 1 <= logs.length <= 100000; 1 <= queries.length <= 100000; logs[i].length == 2; 1 <= logs[i][0] <= n; 1 <= logs[i][1] <= 1000000; 1 <= x <= 100000; x < queries[i] <= 1000000.

Solution: Offline Queries + Sliding Window

For a query q, the relevant interval is [q - x, q]. Every query window has the same width, so process queries in increasing order of q. As q increases, both the left and right boundaries of the relevant log window move only to the right.

Sort logs by time. Sort query indices by query time. Maintain activeCount[server], the number of logs for that server currently inside the window, and activeServers, the number of distinct servers with at least one log inside the window.

For each query in sorted order, add logs with time <= q using the right pointer, then remove logs with time < q - x using the left pointer. The answer is n - activeServers. Store it at the original query index.

Complete Java Solution

```
import java.util.*;

class Solution {
    public int[] countServers(int n, int[][] logs, int x, int[] queries) {
        Arrays.sort(logs, (first, second) ->
                Integer.compare(first[1], second[1]));

        Integer[] queryOrder = new Integer[queries.length];
        for (int index = 0; index < queries.length; index++) {
```

```
            queryOrder[index] = index;
        }

        Arrays.sort(queryOrder, (first, second) ->
                Integer.compare(queries[first], queries[second]));

        int[] activeCount = new int[n + 1];
        int activeServers = 0;
        int left = 0;
        int right = 0;

        int[] answer = new int[queries.length];

        for (int queryIndex : queryOrder) {
            int endTime = queries[queryIndex];
            int startTime = endTime - x;

            while (right < logs.length && logs[right][1] <= endTime) {
                int server = logs[right][0];

                if (activeCount[server] == 0) {
                    activeServers++;
                }
                activeCount[server]++;
                right++;
            }

            while (left < right && logs[left][1] < startTime) {
                int server = logs[left][0];

                activeCount[server]--;
                if (activeCount[server] == 0) {
                    activeServers--;
                }
                left++;
            }

            answer[queryIndex] = n - activeServers;
        }

        return answer;
    }
}
```

Correctness

The window for query q contains exactly the logs whose times satisfy q - x <= time <= q. The right pointer adds every log with time <= q, and the left pointer removes every log with time < q - x. Therefore, after both pointers finish, the window contains precisely the logs in [q - x, q].

activeServers counts a server exactly when its activeCount is positive, so n - activeServers is the number of servers with zero logs in the current interval. Processing queries in increasing time order ensures each pointer only moves forward; each log is added once and removed at most once.

Complexity

Let L = logs.length and Q = queries.length. Time is O(L log L + Q log Q), dominated by sorting logs and query indices. The sliding-window sweep is O(L + Q). Auxiliary space is O(n + Q) for activeCount and the query-order array.

Implementation Notes

Sort an Integer[] of query indices rather than sorting queries directly, so each computed answer can be written back to its original position. Use activeCount[server] == 0 checks when adding and removing logs to maintain the distinct-server count correctly. Duplicate logs for the same server and time are handled naturally because the count tracks occurrences, not just presence.